Houssam Boudiar

Full-Stack Engineer, AI & LLM Applications
Algiers, Algeria · +213 560 25 65 64 · houssam.boudiar0107@gmail.com
Portfolio · Case studies · LinkedIn · GitHub

SUMMARY

Full-stack engineer with 4+ years of experience shipping on-premise products for the oil & gas and public-transport sectors. I build Angular and React frontends, NestJS and FastAPI backends, and LLM agents with LangGraph, including an AI assistant that runs on local models at offline sites. I focus on secure access with Keycloak, real-time data and thorough automated testing.

SKILLS

AI / LLM: LangGraph, LangChain, RAG, pgvector, tool calling, human-in-the-loop, Ollama, vLLM, Azure OpenAI, Anthropic, Gemini
Frontend: Angular, React, TypeScript, RxJS, TanStack, Tauri
Backend: Python, FastAPI, Node.js, NestJS, PostgreSQL, MongoDB, REST, OpenAPI, SSE, MQTT, OPC UA
Security & DevOps: Keycloak, OAuth2 / OpenID Connect, RBAC, Docker, CI/CD
Testing: Playwright, Testcontainers, accessibility testing (WCAG 2.1 AA)
Languages: Arabic (native), English (C1, IELTS), French (B2), German (A2)

EXPERIENCE

Full-Stack Developer, Smartest, Algiers — Mar 2022 – Present

Well Integrity Management System — *oil & gas, on-premise*

- Engineers at remote, offline sites had no quick way to query well data, so I built an AI assistant with LangGraph and FastAPI that answers plain-language questions through read-only tools limited to each user's permissions. It runs on a local Qwen3 model, with Azure OpenAI or Anthropic as fallback, so it works without internet and never exposes data beyond a user's role.
- Operators needed live pressure data for safety decisions. I connected the platform to SCADA over OPC UA and streamed readings to the browser, with limit alarms that must be acknowledged, so every alarm and response is traceable for audit.
- Pressure problems were only caught in manual reviews. I wrote rule-based checks that scan pressure and test history every 15 minutes and feed the risk model, so risk ratings and inspection due dates update automatically when new data arrives.
- The legacy Angular 11 app had become hard to extend. I led its rewrite in React 19 with a NestJS and PostgreSQL backend and Keycloak single sign-on, delivering 12 integrity workflows, GIS maps and 5 languages, including Arabic.

Mobility One — *public-transport ticketing, near completion*

- The backend team's Spring Boot API wasn't ready, so I set up a TypeScript monorepo with shared validation rules, a typed client generated from OpenAPI and a stand-in NestJS backend that follows the same conventions. The Angular frontend moved ahead on its own, and switching to the real backend only needs a URL change.
- Ticket offices must only use approved computers. I built a Windows desktop app with Tauri that ties each session to a key in the PC's TPM chip, so a login can't be reused from any other machine.
- Built operator and user management for metro, tram, bus and cable-car operators, with Keycloak sign-in, per-operator roles in PostgreSQL and rules such as an operator never losing its last admin. Every change is written to an audit log.
- The product runs on-premise and offline, so bugs and licensing problems are costly once shipped. I wrote 170+ automated tests, including 50 Playwright end-to-end tests with accessibility and Arabic layout checks, and CI gates that block risky dependencies.

PROJECTS

AI Trading Signal Agent — *Python, LangGraph, FastAPI, PostgreSQL/pgvector, Docker*

- Built a multi-agent system that analyses gold and Nasdaq charts and suggests trades with entry, stop-loss and take-profit levels, with separate agents for data, analysis and decisions.
- Single-run LLM agents kept repeating the same mistakes, so I added a loop that reviews past signal outcomes and stores the lessons in pgvector. Later analyses pull in the relevant lessons, and a backtesting engine measures results on historical data.

Support-Ops AI Agent — *Python, FastAPI, LangGraph, Gemini, React*

- A support agent that changes customer accounts can't be trusted to act alone, so I built one that proposes actions like trial extensions or credits and pauses until a person approves. No database write can happen without approval.
- Moved spending limits out of the prompt and into code, so the model can't talk its way past them, and added a fallback classifier that labels its results, so a failed model call never looks like a successful one.

EDUCATION

MSc, Software Engineering, University of Constantine 2 – Abdelhamid Mehri, 2020
BSc, Computer Science, University of Constantine 2 – Abdelhamid Mehri, 2018